



Compiladores

Aula 1: Apresentação da disciplina

Prof. Alex Torquato Souza Carneiro
alex.carneiro@impacta.edu.br

Agenda

- Apresentação da disciplina
 - Objetivos
 - Avaliações
 - Bibliografia

- Introdução aos Compiladores
 - Histórico
 - Conceitos Iniciais

Objetivos

- Aprofundar os conhecimentos sobre os fundamentos da computação e linguagens de programação.
- Conhecer a arquitetura e conceitos implementados em compiladores e interpretadores.
- Desenvolver e integrar os componentes principais de um compilador.

Avaliações

- APs
 - Projeto MINIC: 2 etapas por AP
- Prova oficial
- Prova substitutiva

Bibliografia

AHO, A. V.; SETHI, R.; ULLMAN, J. D.
Compiladores: princípios, técnicas e
ferramentas. São Paulo: LTC, 1992.

WATT, D. A., BROWN, D. F., Programming
Language Processors in Java: Compilers and
Interpreters, Prentice-Hall, 2000

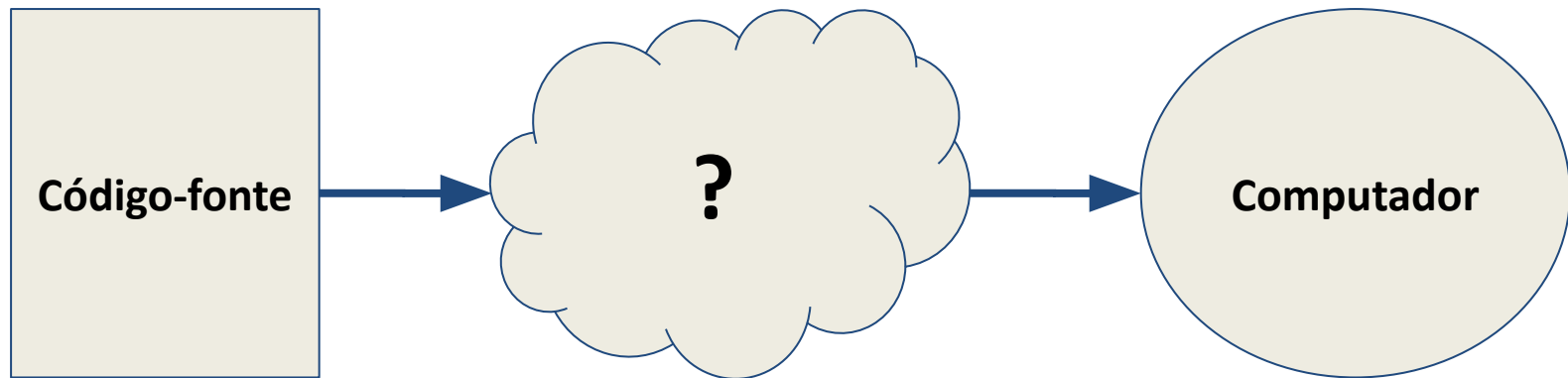
PRICE, A. M. A., TOSCANI, S. S., Implementação
de Linguagens de Programação: Compiladores,
Sagra-Luzzatto, 3a. ed, 2005.

Apresentação da disciplina

- Motivação
 - O que faz um compilador?
 - Elementos básicos de um compilador
- Fundamentos
 - Linguagens de programação
 - Paradigmas de programação
 - Teoria da Computação
- Compilador MINIC
 - Linguagem
 - Elementos
 - Projeto

Motivação

Como transformar um código-fonte em um programa executável?



Motivação

Essa transformação depende de várias etapas:

- Análise léxica;
- Análise de sintaxe;
- Análise de semântica;
- Produção de códigos intermediários;
- Otimização;
- Geração do código executável.

Motivação

A análise léxica trata dos elementos individuais do código, tais como:

- números;
- strings;
- palavras-chave;
- símbolos.

Motivação

A análise de sintaxe trata da organização dos elementos ao longo do código, por exemplo:

- abertura e fechamento de elementos pareados;
- estrutura dos controles de fluxo (while, for, if);
- declarações de estruturas de dados e funções;
- operações.

Motivação

A análise semântica trata das regras de tipo e contexto, por exemplo:

- atribuições de dados;
- elementos necessários a estruturas de dados e/ou funções;
- tipos de dados em operações.

Motivação

A análise semântica trata das regras de tipo e contexto, por exemplo:

- atribuições de dados;
- elementos necessários a estruturas de dados e/ou funções;
- tipos de dados em operações.

Fundamentos

Podemos pensar a execução do código-fonte em de três formas:

- Compilador (C / C++)
- Interpretador (Python, Ruby)
- Modelo híbrido (Java)

Fundamentos

Front-end: Conhece a linguagem

- Scanner e parser
- AST, tabela de símbolos e tipos
- Representação intermediária

Back-end: Conhece a máquina

- Otimização da representação intermediária
- Seleção de instruções
- Registradores e código-alvo



Fundamentos

Exemplo:

x = a + b * c;

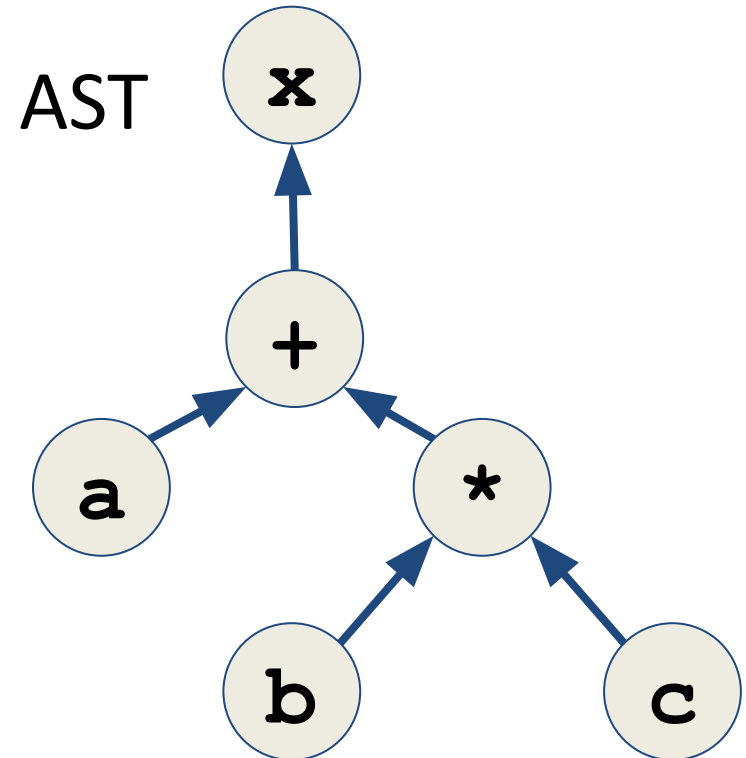
Tokens: {x, =, a, +, b, *, c}

IR:

t1 = b * c

t2 = a + t1

x = t2



Compilador MINIC

MINIC é um compilador para a linguagem C, o qual usaremos em nossa disciplina para fins de estudo e como referência para um projeto.

Etapa 1 Léxico: tokens	Etapa 2 Sintaxe: AST
Etapa 3 Semântico: representação intermediária (IR)	Etapa 4 Código: otimização

Compilador MINIC

Exemplo de código

```
int main() {
```

```
    int total = 2 + 3 * 4;
```

```
    print(total);
```

```
    return 0;
```

```
}
```

1. Quais são os tokens?

2. A expressão é sintaticamente válida?

3. Que informações a semântica precisa verificar?

4. Como representar a execução em etapas?

Compilador MINIC

Exemplo de código

```
int main() {
```

```
    int total = 2 + 3 * 4;
```

```
    print(total);
```

```
    return 0;
```

```
}
```

1. Quais são os tokens?

2. A expressão é sintaticamente válida?

3. Que informações a semântica precisa verificar?

4. Como representar a execução em etapas?

Compilador MINIC

1. Quais são os tokens?

Lexema	Token	Lexema	Token
int	Palavra reservada <code>INT</code>	*	<code>MULTIPLY</code>
main	Identificador <code>IDENTIFIER</code>	4	Literal inteiro <code>INTEGER</code>
(	<code>LPAREN</code>	;	<code>SEMICOLON</code>
)	<code>RPAREN</code>	print	Palavra reservada ou identificador de função
{	<code>LBRACE</code>	(	<code>LPAREN</code>
int	Palavra reservada <code>INT</code>	total	Identificador <code>IDENTIFIER</code>
total	Identificador <code>IDENTIFIER</code>	)	<code>RPAREN</code>
=	<code>ASSIGN</code>	;	<code>SEMICOLON</code>
2	Literal inteiro <code>INTEGER</code>	return	Palavra reservada <code>RETURN</code>
+	<code>PLUS</code>	0	Literal inteiro <code>INTEGER</code>
3	Literal inteiro <code>INTEGER</code>	;	<code>SEMICOLON</code>
		}	<code>RBRACE</code>

Compilador MINIC

2. A expressão é sintaticamente válida?

Programa

└─ Função **main**

└─ Tipo de retorno: **int**

└─ Parâmetros: *nenhum*

└─ Corpo

└─ Declaração de **total**

└─ Chamada de **print**

└─ Comando **return**

Compilador MINIC

3. Que informações a semântica precisa verificar?

1. A função `main` foi declarada corretamente;
2. A variável `total` foi declarada antes de ser utilizada;
3. `total` possui o tipo `int`;
4. Os literais 2, 3 e 4 são inteiros;
5. Os operadores `+` e `*` podem ser aplicados a valores inteiros;
6. O resultado da expressão também é um inteiro;
7. O argumento passado para `print` possui tipo compatível;
8. O valor retornado por `main` é um inteiro.

Compilador MINIC

4. Como representar a execução em etapas?

`t1 = 3 * 4`

`t2 = 2 + t1`

`total = t2`

`param total`

`call print`

`return 0`

Compilador MINIC

Otimização

Após a propagação e avaliação de constantes:

```
total = 14
```

Assim, o código otimizado fica:

```
total = 14
```

```
param total
```

```
call print
```

```
return 0
```

Compilador MINIC

O fluxo completo pode ser resumido assim:

